

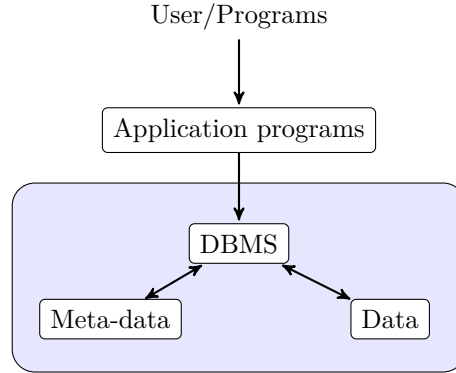
Relational Databases

Weihai Yu

August 2019

1 Relational data model

DBMS



- DBMS is general purpose
- Server or lib

Data abstraction

- In general
 - Containers
 - Data types
 - Bytes
 - Bits
- DB
 - View
 - * Data that metter to particular part of application or user
 - **Logical**

- * Data that matter to the application
- * Physical data independence
- Physical
 - * Data that matter to the hardware or operating system

Tables, relation instances, schemas

instructor(*ID*, *name*, *dept_name*, *salary*)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp.Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000

course(*course_id*, *title*, *dept_name*, *credits*)

<i>course_id</i>	<i>title</i>	<i>dept_name</i>	<i>credits</i>
BIO-101	Intro. Biology	Biology	4
INF-1100	Intro. CS	Comp.Sci.	4
INF-2700	DB	Comp.Sci.	4

- relation name: variable; schema: type; instance: value

Null values

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp.Sci.	65000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000
01010	Newton	<i>null</i>	100000

- null means *unknown*
- or *not applicable* (like middle name)
- thus a source of confusion, *should avoid using it*

Instances, facts, predicates

- Tuples/rows: facts
- Attributes: “things” in facts
- A tuple is a fact *or* a (compound) thing
- The heading of a relation is a *predicate*

- a generic statement (parameterized) about a portion of the real world
- A tuple is a *proposition* that evaluates to *true*
- A relation instance: known facts
- *Attributes* are a mathematical set of names, *tuples* are a mathematical set of facts
 - no duplications, no ordering, no NULLs
 - SQL is a practical approximation
 - * allows NULLs

Keys

We would like to refer to certain facts (this, that, these, ...)

- A *key* of a relation is a set of attributes whose (combined) values uniquely identify tuples
- Superkey: *Instructor*: $ID, \{name, dept_name\}, \{ID, name\}$
- Candidate key (minimal set of attributes): *Instructor*: $ID, \{name, dept_name\}$
- Primary key (one of candidate keys): *Instructor*: ID
- Schema represented as: *instructor*($\underline{ID}, name, dept_name, salary$)

Foreign keys

department($\underline{dept_name}, building$)

$dept_name$	$building$
Comp.Sci.	NT
Finance	HM
Biology	NT

course($\underline{course_id}, title, dept_name, credits$)

$course_id$	$title$	$dept_name$	$credits$
BIO-101	Intro. Biology	Biology	4
INF-1100	Intro. CS	Comp.Sci.	4
INF-2700	DB	Comp.Sci.	4

prereq($\underline{course_id}, \underline{prereq_id}$)

$course_id$	$prereq_id$
INF-2700	INF-1100

- Now think again relations in terms of facts and things.

Query language SQL

instructor(ID, name, dept_name, salary)

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp.Sci.	65000
10102	Gates	Comp.Sci.	10000000
10104	Jobs	Comp.Sci.	1000000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000

select name **from** instructor **where** dept_name = 'Comp.Sci' **and**
salary > 100000;

<i>name</i>
Gates
Jobs

SQL schema

<i>instructor</i> (<u>ID</u> , name, dept_name, salary)
--

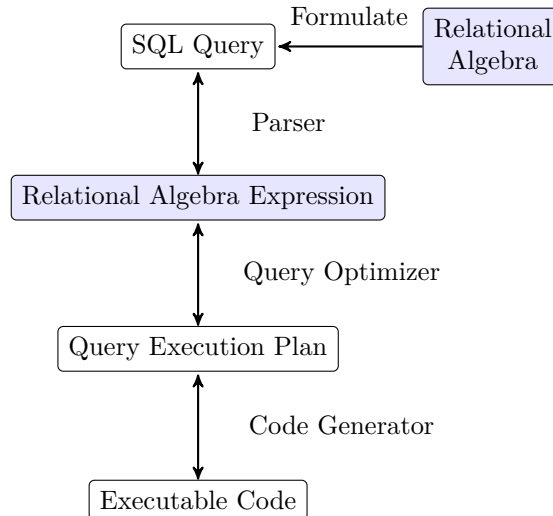
```
create table instructor
(ID varchar(5),
 name varchar(20) not null,
 dept_name varchar(20),
 salary numeric(8,2),
 primary key(ID),
 foreign key(dept_name) references department);
```

2 Relational algebra and SQL

Algebra

- Values
- Operations
- Expressions
- Variables, declarative equations
- Procedural evaluation
- Closure property
 - Relation instances as values for both input and output

Role of relational algebra in DBMS



Select

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp.Sci.	65000
10102	Gates	Comp.Sci.	10000000
10104	Jobs	Comp.Sci.	1000000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000

$\sigma_{dept_name='Comp. Sci' \wedge salary \geq 100000}(instructor)$

select * **from** instructor **where** dept_name = 'Comp.Sci' **and** salary >= 100000;

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10102	Gates	Comp.Sci.	10000000
10104	Jobs	Comp.Sci.	1000000

important: attributes in **where** are of the same single tuples!

Project

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp.Sci.	65000
10102	Gates	Comp.Sci.	10000000
10104	Jobs	Comp.Sci.	1000000
12121	Wu	Finance	90000
15151	Mozart	Music	40000

$$\Pi_{ID, name, salary}(instructor)$$

select distinct ID, name, salary **from** instructor;

<i>ID</i>	<i>name</i>	<i>salary</i>
10101	Srinivasan	65000
10102	Gates	10000000
10104	Jobs	1000000
12121	Wu	90000
15151	Mozart	40000

Notice that in relational algebra, there are no duplicate tuples, but in SQL there are. Therefore we should almost always use **distinct**.

Expression

<i>ID</i>	<i>name</i>	<i>dept_name</i>	<i>salary</i>
10101	Srinivasan	Comp.Sci.	65000
10102	Gates	Comp.Sci.	10000000
10104	Jobs	Comp.Sci.	1000000
12121	Wu	Finance	90000
15151	Mozart	Music	40000
22222	Einstein	Physics	95000

$$\Pi_{name}(\sigma_{dept_name='Comp. Sci' \wedge salary \geq 100000}(instructor))$$

select distinct name **from** instructor **where** dept_name = 'Comp.Sci'
and salary > 100000;

<i>name</i>
Gates
Jobs

Cartesian product

<i>ID</i>	<i>name</i>	<i>dept_n</i>
101	Srinivasan	Comp.Sci.
121	Wu	Finance

<i>course_id</i>	<i>title</i>	<i>dept_n</i>
BIO-101	Intro. Bio	Biology
INF-1100	Intro. CS	Comp. Sci.
INF-2700	DB	Comp. Sci.

$$instructor \times course$$

<i>ID</i>	<i>name</i>	<i>i.dept_n</i>	<i>course_id</i>	<i>title</i>	<i>c.dept_n</i>
101	Srinivasan	Comp.Sci.	BIO-101	Intro. Bio	Biology
101	Srinivasan	Comp.Sci.	INF-1100	Intro. CS	Comp.Sci.
101	Srinivasan	Comp.Sci.	INF-2700	DB	Comp.Sci.
121	Wu	Finance	BIO-101	Intro. Bio	Biology
121	Wu	Finance	INF-1100	Intro. CS	Comp.Sci.
121	Wu	Finance	INF-2700	DB	Comp.Sci.

select * from instructor, course;

dept_n in the two relations are regarded as different “things” and are re-named.

Natural-Join

<i>ID</i>	<i>name</i>	<i>dept_n</i>
101	Srinivasan	Comp.Sci.
102	Gates	Comp.Sci.
121	Wu	Finance

<i>course_id</i>	<i>title</i>	<i>dept_n</i>
BIO-101	Intro. Bio	Biology
INF-1100	Intro. CS	Comp. Sci.
INF-2700	DB	Comp. Sci.

$\boxed{\text{instructor} \bowtie \text{course}}$

<i>ID</i>	<i>name</i>	<i>dept_n</i>	<i>course_id</i>	<i>title</i>
101	Srinivasan	Comp.Sci.	INF-1100	Intro. CS
101	Srinivasan	Comp.Sci.	INF-2700	DB
102	Gates	Comp.Sci.	INF-1100	Intro. CS
102	Gates	Comp.Sci.	INF-2700	DB

select * from instructor **natural join** course;

Now dept_n in the two relations are regarded as the same “thing”.

Theta join

- General form: $\boxed{r \bowtie_{\theta} s = \sigma_{\theta}(r \times s)}$
- With relations schemas:
 $\text{instructor}(\underline{ID}, \text{name}, \text{dept_name}, \text{salary})$
 $\text{teaches}(\text{instructor_id}, \text{course_id}, \text{semester}, \text{year})$
- $\text{instructor} \bowtie_{\text{instructor.ID=teaches.instructor_id}} \text{teaches}$
- **select * from** instructor, teaches **where** instructor.ID = teaches.instructor_id;

Rename

$$\boxed{\rho_X(E)} \quad \text{or} \quad \boxed{\rho_X(A_1, A_2, \dots, A_n)(E)}$$

$\rho_{rich_instructor}(name, salary) (\Pi_{name, salary}(\sigma_{dept_name = \text{“Comp. Sci”} \wedge salary \geq 100000}(instructor)))$

```
with rich_instructor(name,salary) as
(select name, salary
 from instructor
 where dept_name = 'Comp.Sci.' and
 salary >= 100000)
select ... from rich_instructor where ...
```

Various renames in SQL:

```
select name as instructor_name
from   instructor as I, teaches as T
where  I.ID = T.instructor_id;
```

as is not included in earlier versions of SQL. It can be omitted in some systems. Oracle even does not allow it in **from** clauses.

Self join

- What is this?

$teaches \bowtie_{teaches.instructor_id=teaches2.instructor_id} \rho_{teaches2}(teaches)$

- two facts about the same instructors' teaching
- And this?

```
select distinct T1.instructor_id
from   teaches as T1, teaches as T2
where  T1.instructor_id = T2.instructor_id
       and T1.course_id <> T2.course_id;
```

- Instructors that teach at least two different courses.
- All the courses taught in both spring 2011 and fall 2010.
- Remember *the-same-single-tuple* rule?

- ```
select T1.course_id
from teaches T1, teaches T2
where T1.year = 2011 and T1.semester = 'spring'
 and T2.year = 2010 and T2.semester = 'fall'
 and T1.course_id = T2.course_id;
```



## Assignment

- $x \leftarrow v$
- For writing more readable expressions

## Example

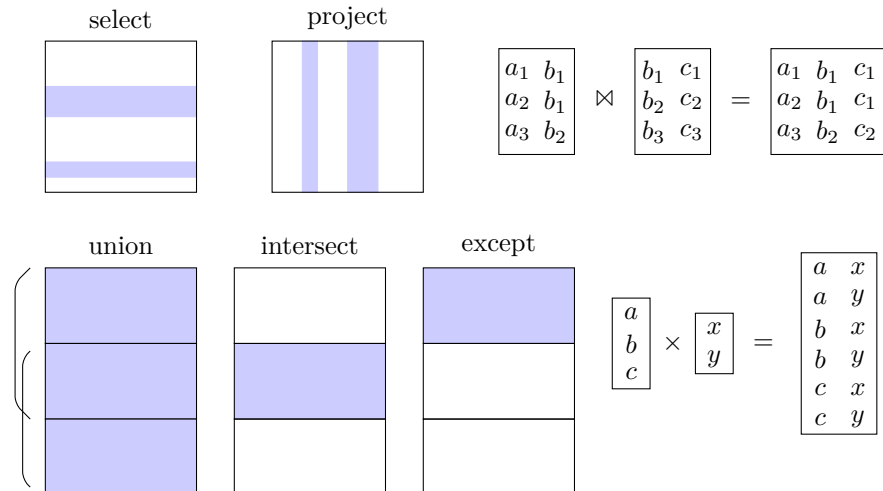
- $\Pi_{A_1}(\sigma_\theta(R \times S))$
- $R\_and\_S \leftarrow R \times S$   
 $R\_and\_S\_with\_ \theta \leftarrow \sigma_\theta(R\_and\_S)$   
 $result \leftarrow \Pi_{A_1}(R\_and\_S\_with\_ \theta)$

## Set operations

- $\cup, \cap, -$  for *compatible relations*.
- **union, union all, intersect, intersect all, except, except all**  
 The **all** version retains all duplicates.
- **in, exists**
- **> all, <> some**

Note that  $r \cap s$  is the same as  $r \bowtie s$

## Operations visualized



## Compound queries

- All the courses taught in both spring 2011 and fall 2010 (again).
- $S11 \leftarrow \sigma_{year=2011 \wedge semester="spring"}(teaches)$   
 $F10 \leftarrow \sigma_{year=2010 \wedge semester="fall"}(teaches)$   
 $\Pi_{course\_id}(S11) \cap \Pi_{course\_id}(F10)$
- ```
select course_id
from teaches
where year = 2011 and semester = 'spring'
intersect
select course_id
from teaches
where year = 2010 and semester = 'fall';
```

Nested subqueries

Names of all instructors whose salary is greater than at least one instructor in the Comp.Sci. department.

```
select name
from instructor
where salary > some
    (select salary
     from instructor
     where dept_name = 'Comp.Sci.');
```

Nested subqueries

- Titles of all courses taught in fall semesters.
- ```
select distinct title
from course natural join teaches
where semester = 'fall';
```
- Titles of all courses *not* taught in fall semesters.
- ```
select distinct title
from course
where course_id not in
    (select course_id
     from teaches
     where semester = 'fall');
```

Correlated nested subqueries

- Courses taught in both fall 2010 and spring 2011 semesters (again).
- ```
select distinct course_id
from teaches as T
where semester = 'fall' and year = 2010 and
exists
 (select *
 from teaches
 where semester = 'spring' and year = '2011'
 and couse_id = T.course_id);
```
- *Expensive!*
  - The inner-subquery is executed for every row of the outer-subquery.

### R contains S

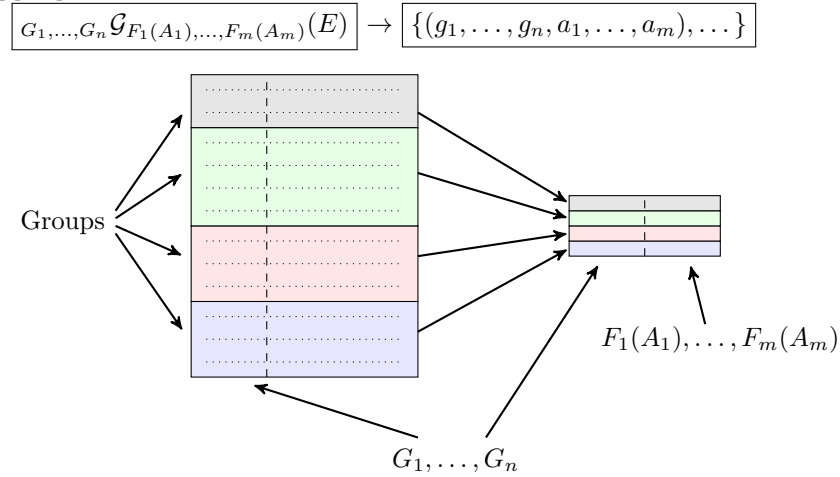
- $\boxed{R \supseteq S} = \boxed{(S - R) = \emptyset}$
- **not exists** (S except R)
- All students who have taken all courses offered by Comp.Sci department.
- ```
select distinct S.name
from   student as S
where  not exists
      (select course_id
       from   course
       where  dept_name = 'Comp.Sci.')
except
      (select course_id
       from   takes
       where  S.ID = ID);
```

Aggregates

- $\boxed{G_1, G_2, \dots, G_n \mathcal{G}_{F_1(A_1), F_2(A_2), \dots, F_m(A_m)}} \rightarrow \boxed{\{(g_1, g_2, \dots, g_n, a_1, a_2, \dots, a_m), \dots\}}$
- G_i : attributes for grouping, F_j : aggregate function on attribute A_j
- $\boxed{\mathcal{G}_{\text{sum}(\text{salary})}(\text{instructor})}$
- $\text{sum}(10000, 10000, 3000) \rightarrow 23000$
- **sum, avg, count, max, min**
- **sum(distinct), avg(distinct), count(distinct), max(distinct), min(distinct)**

- $\boxed{\text{dept_name} \mathcal{G}_{\text{avg}(\text{salary})}(\text{instructor})}$
- **select** dept_name, **avg**(salary) **as** avg_salary **from** instructor **group**
by dept_name;

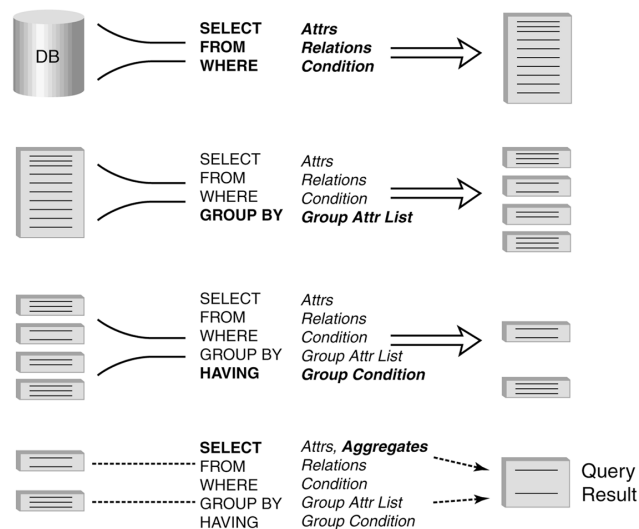
Aggregates



Having

- **select** dept_name, **avg**(salary) **as** avg_salary **from** instructor **group**
by dept_name **having** **avg**(salary) > 10000;
- Instructors that teach at least two different courses (again).
select instructor_id **from** Teaches **group by** instructor_id **having**
count(**distinct** course_id) >= 2;

Group by, having



String

- `dept_name = 'Comp.Sci.'`
- `course_name like 'INF%'`

Order by, limit

```
select *
from   instructor
order by dept_name asc, salary desc
limit 5;
```

What is this?

```
select *
from   instructor natural join instructor;
```

- the same instructor instance
- always
- really?
 - What if `dept_name` is null?

Null values

- Value is *unknown* or *not applicable*, but it is *not false*
- Any comparison with null yields null.
 - $1 < \text{null} \rightarrow \text{null}$
 - $\text{salary} = \text{null} \rightarrow \text{null}$
 - $\text{null} = \text{null} \rightarrow \text{null}$
- Instead, use **is null**, **is not null**
- *But* in **select distinct**: $\text{null} = \text{null}$
- Three-value logic:

	○	true	false	null
○ and	null	null	false	null
○ or	null	true	null	null
not ○		false	true	null

Outer joins

- Display all students, their IDs, names, etc., together with the courses they have taken.
- **select * from student natural join** takes;
- Wait, what about the students who take no course?
- **select * from student natural left outer join** takes;

Delete, insert, update, grant

```
delete from instructor where dept_name = 'NT-Fak';
insert into course values ('INF-2701', 'Fun DB', 'Comp.Sci.', 4);
update instructor set salary = salary * 1.05 where salary < 3000;
grant update (grade) on takes to karin, jan;
```

Views

```
create view dept_total_salary(dept_name, total_salary) as select dept_name,
sum(salary) from instructor group by dept_name;
```

3 Integrity constraints

Integrity constraints

Restrictions on

- database state (static)
 - Intra- and inter-relations
 - enforced by DBMS
- sequences of states (dynamic)
 - enforced by application programs

Intra-relational constraints

```
create table course (  
    course_id varchar (8),  
    title varchar (50), not null,  
    dept_name varchar (20),  
    credits numeric (2,0), default 4, check (credits > 0),  
    primary key (course_id),  
    unique (title, dept_id),  
    foreign key (dept_name) references department  
        on delete cascade  
        on update cascade);
```

Inter-relational constraints

create assert max_instructor_load check

```
(not exists (select *  
    from      course natural join teaches  
    group by ID  
    having    sum(credits) > 10));
```